Plugins

Quick Start

A plugin is just a **small code module** that extends OpenClaw with additional capabilities (the commands, the tools and the Gateway RPCs). Most of the time, you use a plugin when you want a capability that is not built into the core of OpenClaw (or when you want to keep the optional capabilities out of the main installation). The quick path:

1. Look at what is loaded:

```
openclaw plugins list
```

1. Install an official plugin (for example: Voice Call):

```
openclaw plugins install @openclaw/voice-call
```

1. Restart the Gateway and then configure it under `plugins.entries.<id>.config`.

The Available Plugins (official)

- Since 2026.1.15 Microsoft Teams is only offered as a plugin; if you use Teams, install `@openclaw/msteams`.
- Memory (Core) — the bundled memory search plugin (enabled by default through `plugins.slots.memory`)
- Memory (LanceDB) — the bundled long term memory plugin (the automatic recall/capture; set `plugins.slots.memory = "memory-lancedb"`)
- Voice Call — `@openclaw/voice-call`
- Zalo Personal — `@openclaw/zalouser`
- Matrix — `@openclaw/matrix`
- Nostr — `@openclaw/nostr`
- Zalo — `@openclaw/zalo`
- Microsoft Teams — `@openclaw/msteams`
- Google Antigravity OAuth (the provider authentication) — bundled as `google-antigravity-auth` (disabled by default)
- Gemini CLI OAuth (the provider authentication) — bundled as `google-gemini-cli-auth` (disabled by default)
- Qwen OAuth (the provider authentication) — bundled as `qwen-portal-auth` (disabled by default)
- Copilot Proxy (the provider authentication) — the local VS Code Copilot Proxy bridge; it is different from the built in `github-copilot` device login (bundled, disabled by default)

The OpenClaw plugins are **TypeScript modules** that are loaded at runtime through jiti. **The configuration validation does not execute the plugin code**; it uses the plugin manifest and the JSON Schema. Refer to [the plugin manifest]. A plugin can register:

- The Gateway RPC methods
- The Gateway HTTP handlers
- The agent tools
- The CLI commands
- The background services
- The optional configuration validation
- **The Skills** (by listing a `skills` directory in the plugin manifest)
- **The auto reply commands** (executed without calling the AI agent)

The plugins run in the **same process** as the Gateway, so treat them as trusted code. The guide for writing the tools: [the plugin agent tools].

The Runtime Helper

A plugin can access the selected core helper tools through `api.runtime`. For the telephony TTS:

```
const result = await api.runtime.tts.textToSpeechTelephony({
  text: "Hello from OpenClaw",
  cfg: api.config,
});
```

The points to note:

- It uses the core `messages.tts` configuration (OpenAI or ElevenLabs).
- It returns a PCM audio buffer + the sample rate. The plugin has to resample/encode it for the provider.
- The Edge TTS does not support the telephony.

The Discovery and the Priority

OpenClaw scans in the following order:

1. The configuration paths

- `plugins.load.paths` (a file or a directory)

1. The workspace extensions

- `<workspace>/.openclaw/extensions/*.ts`
- `<workspace>/.openclaw/extensions/*/index.ts`

1. The global extensions

- `~/.openclaw/extensions/*.ts`
- `~/.openclaw/extensions/*/index.ts`

1. The bundled extensions (published together with OpenClaw, **disabled by default**)

- `<openclaw>/extensions/*`

The bundled plugins have to be enabled explicitly through `plugins.entries.<id>.enabled` or `openclaw plugins enable <id>`. The installed plugins are enabled by default, but they can be disabled in the same way. Every plugin has to contain an `openclaw.plugin.json` file in its root directory. If the path points to a file, the root of the plugin is the directory of the file and it has to contain the manifest. If several plugins resolve to the same id, the first match in the order above wins and the copy with the lower priority is ignored.

The Package Collections

A plugin directory can contain a `package.json` with `openclaw.extensions`:

```
{
  "name": "my-pack",
  "openclaw": {
    "extensions": ["./src/safety.ts", "./src/tools.ts"]
  }
}
```

Each entry becomes a plugin. If the package lists several extensions, the plugin id becomes `name/<fileBase>`. If your plugin imports the npm dependencies, install them in that directory so that `node_modules` is available (`npm install` / `pnpm install`).

The Channel Catalogue Metadata

A channel plugin can broadcast the onboarding metadata through `openclaw.channel` and the installation hints through `openclaw.install`. This keeps the core catalogue free of data. An example:

```
{
  "name": "@openclaw/nextcloud-talk",
  "openclaw": {
    "extensions": ["./index.ts"],
    "channel": {
      "id": "nextcloud-talk",
      "label": "Nextcloud Talk",
      "selectionLabel": "Nextcloud Talk (self-hosted)",
      "docsPath": "/channels/nextcloud-talk",
      "docsLabel": "nextcloud-talk",
      "blurb": "Self-hosted chat via Nextcloud Talk webhook bots.",
      "order": 65,
      "aliases": ["nc-talk", "nc"]
    },
    "install": {
      "npmSpec": "@openclaw/nextcloud-talk",
      "localPath": "extensions/nextcloud-talk",
      "defaultChoice": "npm"
    }
  }
}
```

OpenClaw can also merge the **external channel catalogues** (for example the exports of the MPM registry). Put the JSON file in one of the following places:

- `~/.openclaw/mpm/plugins.json`
- `~/.openclaw/mpm/catalog.json`
- `~/.openclaw/plugins/catalog.json`

Or point `OPENCLAW_PLUGIN_CATALOG_PATHS` (or `OPENCLAW_MPM_CATALOG_PATHS`) to one or more JSON files (comma/semicolon/`PATH` separated). Each file should contain `{ "entries": [ { "name": "@scope/pkg", "openclaw": { "channel": {...}, "install": {...} } } ] }`.

The Plugin IDs

The default plugin ids:

- The package collections: the `name` of the `package.json`
- The standalone files: the base name of the file (`~/.../voice-call.ts` → `voice-call`)

If a plugin exports an `id`, OpenClaw uses it, but it issues a warning when it does not match the configured id.

The Configuration

```
{
  plugins: {
    enabled: true,
    allow: ["voice-call"],
    deny: ["untrusted-plugin"],
    load: { paths: ["~/Projects/oss/voice-call-extension"] },
    entries: {
      "voice-call": { enabled: true, config: { provider: "twilio" } },
    },
  },
}
```

The fields:

- `enabled`: the master switch (the default: true)
- `allow`: the allow list (optional)
- `deny`: the deny list (optional; deny has the priority)
- `load.paths`: the additional plugin files/directories
- `entries.<id>`: the switch + the configuration of each plugin

A configuration change **requires a restart of the Gateway**. The validation rules are strict:

- An unknown plugin id in `entries`, `allow`, `deny` or `slots` is an **error**.
- An unknown `channels.<id>` key is an **error**, unless the plugin manifest declares the channel id.
- The plugin configuration is validated with the JSON Schema that is embedded in `openclaw.plugin.json` (`configSchema`).
- If a plugin is disabled, its configuration is kept and a **warning** is issued.

The Plugin Slots (the exclusive categories)

Some of the plugin categories are **exclusive** (only one is active at a time). Use `plugins.slots` to select the plugin that owns the slot:

```
{
  plugins: {
    slots: {
      memory: "memory-core", // or "none" to disable the memory plugins
    },
  },
}
```

If several plugins declare `kind: "memory"`, only the selected one is loaded. The others are disabled with a diagnostic message.

The Control Interface (the schema + the labels)

The control interface uses `config.schema` (the JSON Schema + `uiHints`) to render better forms. OpenClaw enhances the `uiHints` at runtime according to the plugins that it discovers:

- It adds the per plugin labels for `plugins.entries.<id>` / `.enabled` / `.config`
- It merges the optional configuration field hints that the plugin provides at the following places: `plugins.entries.<id>.config.<field>`

If you want the configuration fields of your plugin to show nice labels/placeholders (and to mark the secrets as sensitive), provide the `uiHints` and the JSON Schema in the plugin manifest. An example:

```
{
  "id": "my-plugin",
  "configSchema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "apiKey": { "type": "string" },
      "region": { "type": "string" }
    }
  },
  "uiHints": {
    "apiKey": { "label": "API Key", "sensitive": true },
    "region": { "label": "Region", "placeholder": "us-east-1" }
  }
}
```

The CLI

```
openclaw plugins list
openclaw plugins info <id>
openclaw plugins install <path>                 # copy a local file/dir into
~/.openclaw/extensions/<id>
openclaw plugins install ./extensions/voice-call # a relative path is ok
openclaw plugins install ./plugin.tgz           # install from a local tarball
openclaw plugins install ./plugin.zip           # install from a local zip
openclaw plugins install -l ./extensions/voice-call # link (no copy) for dev
openclaw plugins install @openclaw/voice-call # install from npm
openclaw plugins update <id>
openclaw plugins update --all
openclaw plugins enable <id>
openclaw plugins disable <id>
openclaw plugins doctor
```

`plugins update` only applies to the npm installations that are tracked under `plugins.installs`. A plugin can also register its own top level command (for example: `openclaw voicecall`).

The Plugin API (an overview)

A plugin exports one of the following:

- A function: `(api) => { ... }`
- An object: `{ id, name, configSchema, register(api) { ... } }`

The Plugin Hooks

A plugin can ship the hooks and register them at runtime. This lets a plugin bundle the event driven automation without installing the hook packages separately.

An Example

```
import { registerPluginHooksFromDir } from "openclaw/plugin-sdk";

export default function register(api) {
  registerPluginHooksFromDir(api, "./hooks");
}
```

The points to note:

- The hook directory follows the normal hook structure (`HOOK.md` + `handler.ts`).
- The hook eligibility rules still apply (the operating system/binary/environment/configuration requirements).
- The plugin managed hooks appear as `plugin:<id>` in `openclaw hooks list`.
- You cannot enable/disable the plugin managed hooks through `openclaw hooks`; you enable/disable the plugin instead.

The Provider Plugins (the model authentication)

A plugin can register a **model provider authentication** flow, so that the user can run the OAuth or the API key setup inside OpenClaw (without an external script). The provider is registered through `api.registerProvider(...)`. Each provider exposes one or more authentication methods (OAuth, the API key, the device code and so on). These methods drive:

- `openclaw models auth login --provider <id> [--method <id>]`

An example:

```
api.registerProvider({
  id: "acme",
  label: "AcmeAI",
  auth: [
    {
      id: "oauth",
      label: "OAuth",
      kind: "oauth",
      run: async (ctx) => {
        // Run the OAuth flow and return the auth profiles.
        return {
          profiles: [
            {
              profileId: "acme:default",
              credential: {
                type: "oauth",
                provider: "acme",
                access: "...",
                refresh: "...",
                expires: Date.now() + 3600 * 1000,
              },
            },
          ],
          defaultModel: "acme/opus-1",
        };
      },
    },
  ],
});
```

The points to note:

- The `run` receives a `ProviderAuthContext` with the `prompter`, the `runtime`, the `openUrl` and the `oauth.createVpsAwareHandlers` helpers.
- Return a `configPatch` when the default model or the provider configuration has to be added.
- Return a `defaultModel` so that `--set-default` can update the default of the agent.

Registering a Message Channel

A plugin can register a **channel plugin**, which behaves similarly to the built in channels (WhatsApp, Telegram and so on). The channel configuration is under `channels.<id>` and it is validated by your channel plugin code.

```
const myChannel = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "demo channel plugin.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
  config: {
    listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ?? {}),
    resolveAccount: (cfg, accountId) =>
      cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
        accountId,
      },
  },
  outbound: {
    deliveryMode: "direct",
    sendText: async () => ({ ok: true }),
  },
};

export default function (api) {
  api.registerChannel({ plugin: myChannel });
}
```

The points to note:

- Put the configuration under `channels.<id>` (and not under `plugins.entries`).
- `meta.label` is the label that is used in the CLI/UI list.
- `meta.aliases` adds the alternative ids that are used for the normalisation and for the CLI input.
- `meta.preferOver` lists the channel ids to skip the automatic enabling of when both of them are configured.
- `meta.detailLabel` and `meta.systemImage` let the UI show a richer channel label/icon.

Writing a New Message Channel (a step by step guide)

Use this when you want a **new chat interface** (a "message channel") rather than a model provider. The model provider documentation is under `/providers/*`.

1. Choose the id + the structure of the configuration

- All the channel configurations are under `channels.<id>`.

- For a multi account setup, prefer `channels.<id>.accounts.<accountId>`.

1. Define the channel metadata

- `meta.label`, `meta.selectionLabel`, `meta.docsPath`, `meta.blurb` control the CLI/UI list.
- `meta.docsPath` should point to a documentation page such as `/channels/<id>`.
- `meta.preferOver` lets the plugin replace another channel (it is preferred by the automatic enabling).
- `meta.detailLabel` and `meta.systemImage` are used by the UI for the detailed text/icon.

1. Implement the required adapters

- `config.listAccountIds` + `config.resolveAccount`
- `capabilities` (the chat types, the media, the threads and so on)
- `outbound.deliveryMode` + `outbound.sendText` (for the basic sending)

1. Add the optional adapters as needed

- `setup` (the wizard), `security` (the DM policy), `status` (the health/diagnostics)
- `gateway` (the start/stop/login), `mentions`, `threading`, `streaming`
- `actions` (the message actions), `commands` (the native command behaviour)

1. Register the channel in the plugin

- `api.registerChannel({ plugin })`

An example of the minimal configuration:

```
{
  channels: {
    acmechat: {
      accounts: {
        default: { token: "ACME_TOKEN", enabled: true },
      },
    },
  },
}
```

A minimal channel plugin (the outbound only):

```
const plugin = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "AcmeChat messaging channel.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
  config: {
```

```
    listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ?? {}),
    resolveAccount: (cfg, accountId) =>
      cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
        accountId,
      },
  },
  outbound: {
    deliveryMode: "direct",
    sendText: async ({ text }) => {
      // deliver `text` to your channel here
      return { ok: true };
    },
  },
};

export default function (api) {
  api.registerChannel({ plugin });
}
```

Load the plugin (the extensions directory or `plugins.load.paths`), restart the Gateway and then configure `channels.<id>` in the configuration.

The Agent Tools

Refer to the dedicated guide: the plugin agent tools.

Registering a Gateway RPC Method

```
export default function (api) {
  api.registerGatewayMethod("myplugin.status", ({ respond }) => {
    respond(true, { ok: true });
  });
}
```

Registering a CLI Command

```
export default function (api) {
  api.registerCli(
    ({ program }) => {
      program.command("mycmd").action(() => {
        console.log("Hello");
      });
    },
    { commands: ["mycmd"] },
  );
}
```

Registering an Auto Reply Command

A plugin can register a custom slash command that is executed **without calling the AI agent**. This is useful for the toggle commands, the status checks or the quick actions that do not need any LLM processing.

```
export default function (api) {
  api.registerCommand({
    name: "mystatus",
    description: "Show plugin status",
    handler: (ctx) => ({
      text: `Plugin is running! Channel: ${ctx.channel}`,
    }),
  });
}
```

The context of the command handler:

- `senderId`: the ID of the sender (if available)
- `channel`: the channel the command was sent from
- `isAuthorizedSender`: whether the sender is an authorised user
- `args`: the arguments that are passed after the command (if `acceptsArgs: true`)
- `commandBody`: the full text of the command
- `config`: the current OpenClaw configuration

The options of the command:

- `name`: the name of the command (without the leading `/`)
- `description`: the help text that is shown in the command list
- `acceptsArgs`: whether the command accepts the arguments (the default: false). If it is false and the arguments are provided, the command does not match and the message is passed to the other handlers
- `requireAuth`: whether an authorised sender is required (the default: true)
- `handler`: the function that returns `{ text: string }` (it can be asynchronous)

An example with the authorisation and the arguments:

```
api.registerCommand({
  name: "setmode",
  description: "Set plugin mode",
  acceptsArgs: true,
  requireAuth: true,
  handler: async (ctx) => {
    const mode = ctx.args?.trim() || "default";
    await saveMode(mode);
    return { text: `Mode set to: ${mode}` };
  },
});
```

The points to note:

- The plugin commands are handled **before** the built in commands and the AI agent
- The commands are registered globally and they apply to all the channels
- The name of the command is not case sensitive (`/MyStatus` matches `/mystatus`)
- The name of the command has to start with a letter and it may only contain the letters, the digits, the hyphens and the underscores
- The reserved command names (such as `help`, `status`, `reset` and so on) cannot be overridden by a plugin
- A duplicate command registration across the plugins fails with a diagnostic error

Registering a Background Service

```
export default function (api) {
  api.registerService({
    id: "my-service",
    start: () => api.logger.info("ready"),
    stop: () => api.logger.info("bye"),
  });
}
```

The Naming Conventions

- The Gateway methods: `pluginId.action` (for example: `voicecall.status`)
- The tools: `snake_case` (for example: `voice_call`)
- The CLI commands: kebab or camel, but avoid a conflict with the core commands

Skills

A plugin can ship the Skills in the repository (`skills/<name>/SKILL.md`). Enable it with `plugins.entries.<id>.enabled` (or another configuration gate) and make sure that it exists in your workspace/hosted Skills location.

The Distribution (npm)

The recommended packaging:

- The main package: `openclaw` (this repository)
- The plugins: the separate npm packages under `@openclaw/*` (for example: `@openclaw/voice-call`)

The publishing contract:

- The `package.json` of the plugin has to contain `openclaw.extensions` with one or more entry files.
- The entry files can be `.js` or `.ts` (jiti loads the TS at runtime).
- `openclaw plugins install <npm-spec>` uses `npm pack`, extracts it into `~/.openclaw/extensions/<id>/` and enables it in the configuration.
- The stability of the configuration key: a scoped package is normalised to an **unscoped** id of

`plugins.entries.*`.

The Example Plugin: Voice Call

This repository contains a voice call plugin (Twilio or the log fallback):

- The source: `extensions/voice-call`
- The Skills: `skills/voice-call`
- The CLI: `openclaw voicecall start|status`
- The tool: `voice_call`
- The RPC: `voicecall.start`, `voicecall.status`
- The configuration (twilio): `provider: "twilio"` + `twilio.accountSid/authToken/from` (the optional `statusCallbackUrl` and `twimlUrl`)
- The configuration (dev): `provider: "log"` (no network)

Refer to Voice Call and to `extensions/voice-call/README.md` for the setup and the usage.

The Security Notes

The plugins run in the same process as the Gateway. Treat them as trusted code:

- Only install the plugins that you trust.
- Prefer the `plugins.allow` allow list.
- Restart the Gateway after a change.

Testing a Plugin

A plugin can (and should) ship the tests:

- The plugins in the repository can keep the Vitest tests under `src/**` (for example: `src/plugins/voice-call.plugin.test.ts`).
- A separately published plugin should run its own CI (lint/build/test) and verify that `openclaw.extensions` points at the built entry point (`dist/index.js`).

For more about the plugins, refer to: Plugins - OpenClaw